

# The Daily Crumb

MIS-403 Project Presentation: A Click & Collect Bakery Web Application

---

# Project Origins & Design

# Selecting The Concept

For my MIS-403 project, I chose to build a functional 'Click & Collect' web application for a local bakery.

I wanted a project that was structurally simple but required robust, real-world business logic. A bakery model necessitates dynamic preparation time calculations based on inventory, making it the perfect candidate to showcase my problem-solving and front-end development skills.



# Designing the Visual Identity



## Warm Cream

Selected as the primary background. It provides a clean, soft canvas that is far less harsh than pure white, evoking the cozy feel of a bakery.



## Dark Chocolate

Used for all primary typography and headings. It ensures high readability and contrast while maintaining a premium, artisanal aesthetic.



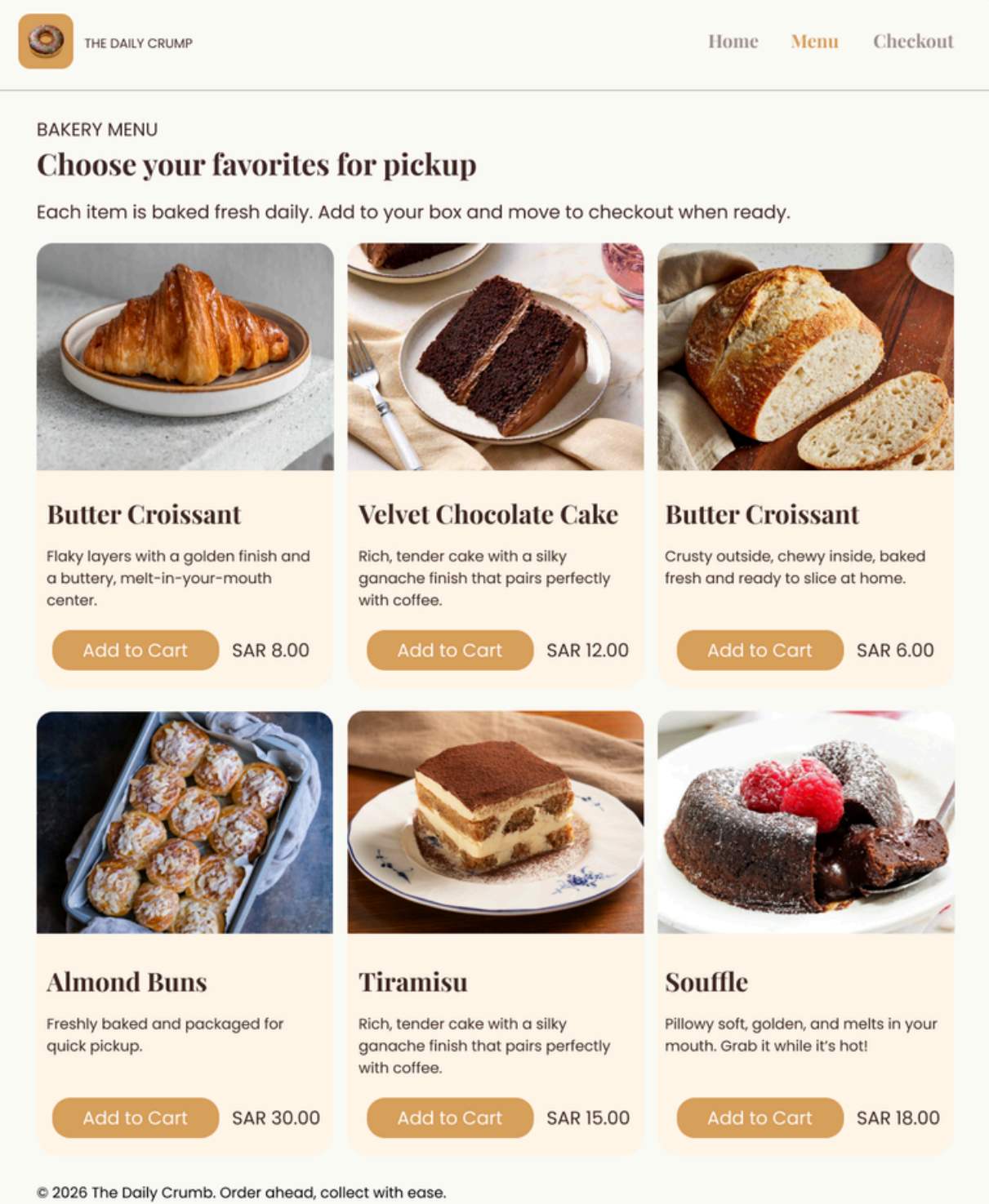
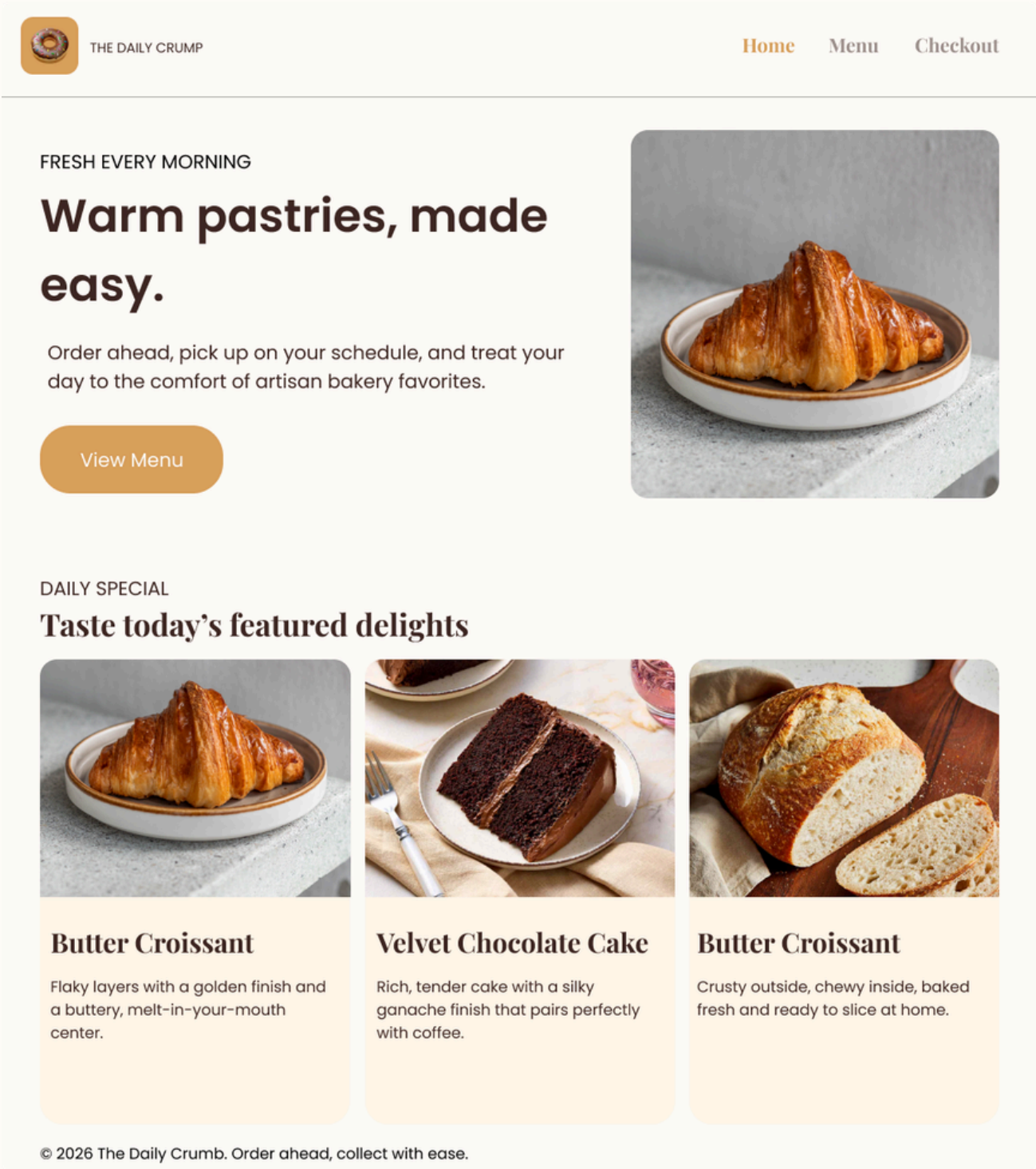
## Honey Gold

Acts as the energetic accent color. It is utilized strategically for buttons and active states to guide the user naturally toward the checkout.

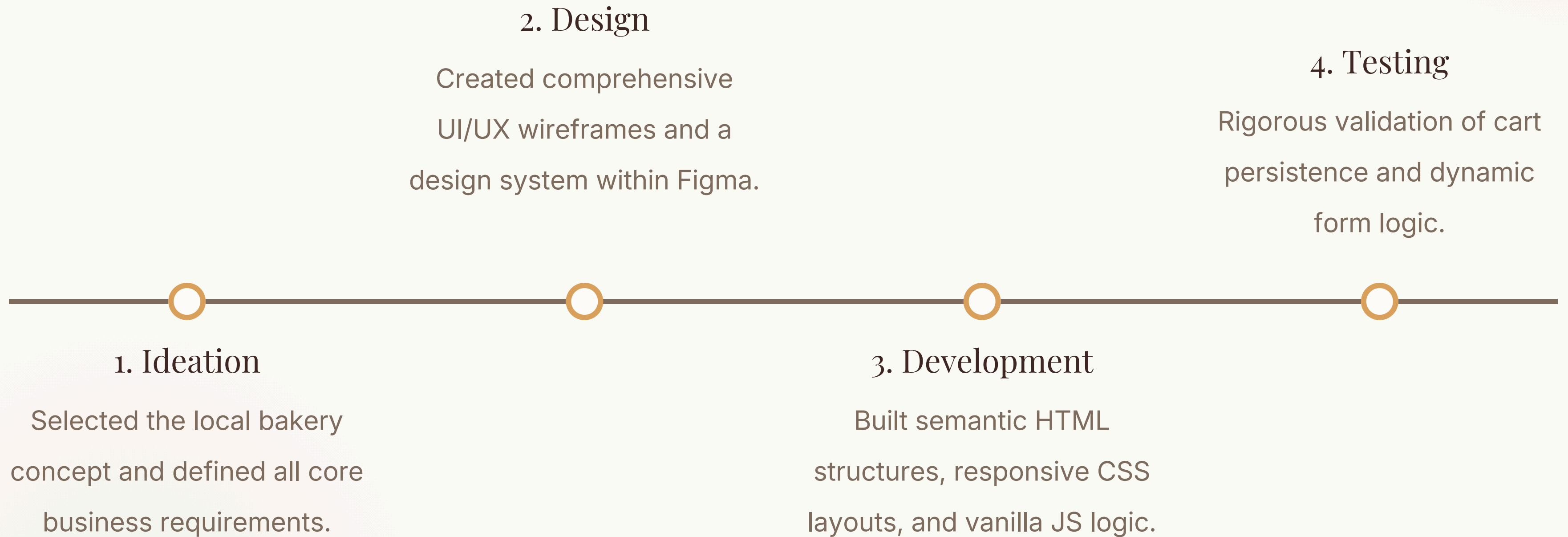
# UI/UX Design Process in Figma

Before writing a single line of code, I architected the entire user journey using Figma.

I mapped out the primary navigation flows between the Home, Menu, and Checkout pages. By developing high-fidelity mockups first, I was able to rapidly test my color palette and typography choices, ensuring a highly intuitive and visually consistent shopping experience.



# Project Development Process



# Core Architecture

## A Clean, Flat File Structure

The application is built cleanly without complex frameworks. 'index.html' is the welcoming landing page. 'menu.html' utilizes a responsive CSS grid for products. 'checkout.html' handles review and scheduling. 'app.js' manages all state, and 'style.css' ensures global consistency.

```
function getCart() {
  const saved = localStorage.getItem(STORAGE_KEY);
  return saved ? JSON.parse(saved) : {};
}

function saveCart(cart) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(cart));
}

function addItemToCart(productId) {
  const cart = getCart();
  cart[productId] = (cart[productId] || 0) + 1;
  saveCart(cart);
}

function removeItemFromCart(productId) {
  const cart = getCart();
  delete cart[productId];
  saveCart(cart);
}

function updateCartQuantity(productId, quantity) {
  const cart = getCart();
  if (quantity < 1) {
    delete cart[productId];
  } else {
    cart[productId] = quantity;
  }
  saveCart(cart);
}

function getCartDetails() {
  const cart = getCart();
  return Object.entries(cart).map(([productId, quantity]) => {
    const item = menuItems.find(product => product.id === productId);
    return {
      ...item,
      quantity
    };
  });
}
```

# Advanced Business Logic

-  **State Persistence:** The shopping cart state is managed entirely using the browser's native `'localStorage'` API, maintaining data smoothly between page loads.
-  **Base Preparation Time:** To simulate realistic kitchen operations, the system enforces a strict base preparation buffer of 15 minutes before any order can be collected.
-  **Dynamic Item Scaling:** The JavaScript logic scales the preparation time dynamically, adding an additional 5 minutes of mandatory wait time for every individual item placed in the cart.
-  **DOM Manipulation & Security:** The script fetches the current real-world time, adds the calculated prep minutes, and dynamically alters the HTML ``min`` attribute on the time picker to prevent impossible scheduling.

# Showcasing the Menu



Artisanal Sourdough



Classic French Soufflé



Warm Butter Croissant

# Development Challenges



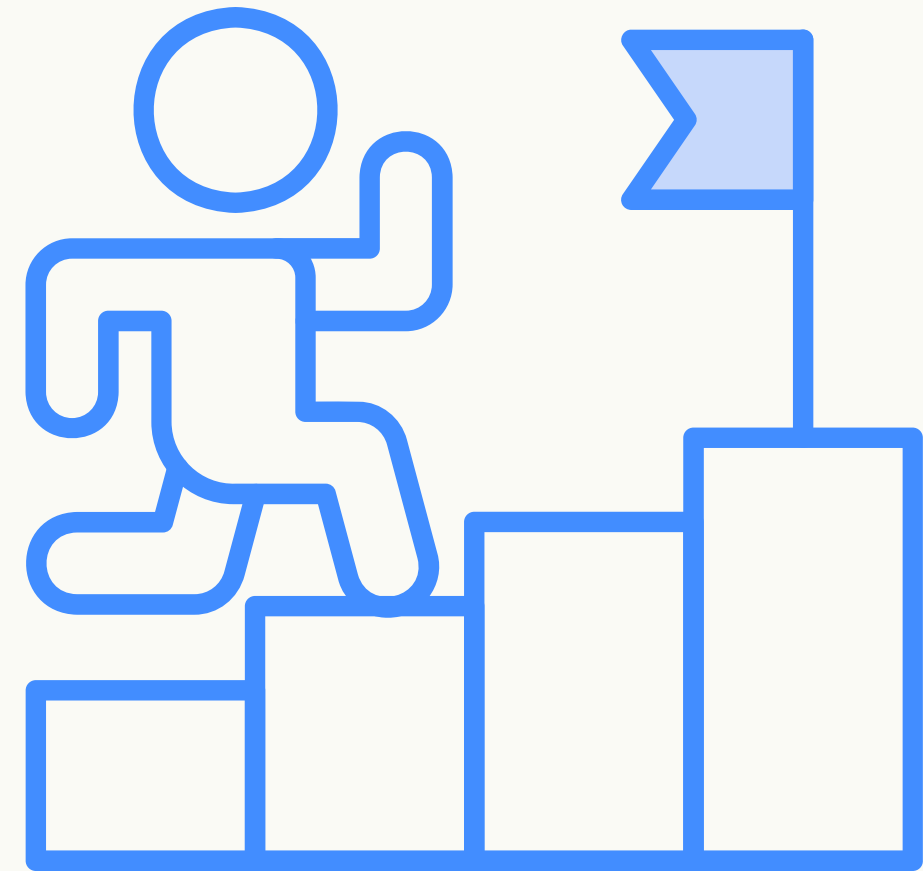
Building this web application entirely from scratch presented several notable technical hurdles.

Managing state across disparate HTML pages without a modern reactive backend framework required precise handling of the `localStorage` object. Furthermore, manipulating the DOM to enforce dynamic form validation for the time picker taught me incredibly valuable lessons regarding vanilla JavaScript event handling, date parsing, and robust error management.

# The Next Steps

While this current prototype successfully demonstrates the core user journey and logic, there is significant room for scaling.

Future iterations of The Daily Crumb application will include full backend database integration for live inventory management, a secure payment gateway for immediate transaction processing, and user account creation to save order history and loyalty points.



# Thank You

MIS-403 Project Presentation | The Daily Crumb